The Context Coupling Problem: Exploration vs Editing

Project report – coding agent traces (SWE-chat), inspired by the SWE-Edit paper

1 What the problem is

A coding agent usually does everything in **one context window**: it reads files, plans changes, and writes the edits all in the same place. The SWE-Edit paper calls this the **context coupling problem**. The trouble is that all the file-reading piles up in the expensive model's context and clogs it. SWE-Edit's fix is to split the work by **role**: a cheap "Viewer" model reads the files and keeps only the useful parts, and a separate "Editor" writes the changes. On their benchmark this cut cost by 17.9% and even improved accuracy by 2.1 points.

My goal for this row was to measure, on real sessions, how big this coupling problem is, and to estimate how much money the SWE-Edit idea could save.

2 What I did

- I selected **long sessions** (more than 30 turns) from Claude Code – the ones big enough to really have this problem.
- I counted how often the agent did each kind of action, using the tool it called (reading, editing, running commands).
- I then measured the **token size** of each action's result – how much text each activity pushes into the context – because reading a file adds far more than making a small edit.

3 What I saw

There are 769 long Claude Code sessions. I found three main activities, not two – so I added a third category, "Execution" (running tests and commands):

Activity	Share of actions	Share of context tokens
Exploration (reading files)	30%	**55%**
Execution (running commands)	38%	19%
Editing (writing code)	20%	2%
Coordination (planning/tasks)	11%	24%

The most important line is the first one. **Exploration is only 30% of the actions, but 55% of all the tokens in the context.** Reading files is by far the biggest thing filling the expensive model's window – which is exactly what SWE-Edit moves to a cheaper model. Editing, on the other hand, is only 2% of tokens, so it is cheap: the Editor part helps quality, not cost.

> **A surprise – Execution:** running commands (Bash) is the most frequent action (38%) and adds 19% of the tokens. The SWE-Edit paper does not focus on this, but in real sessions command output (test logs, errors) is a second thing that fills the context. This is my own extra finding.

4 What I concluded

My measurements show clearly that the coupling problem is real, and that **reading is what drives it**. Exploration takes up **55% of all the tokens in the context** – by far the largest source – while editing takes only 2%. So the expensive model spends most of its context on file-reading, which is exactly the work a cheaper Viewer model could take over. Because reading dominates the context so strongly, that is where the real savings are; splitting off editing barely changes cost, so its value is in reliability, not money.

My key finding: file-reading is 55% of the context in long sessions (~$10,250 of cost) – and offloading it to a cheaper Viewer could save about $6,150.

> **From size to savings:** I measured the size of the problem directly: file-reading is 55% of the context, which works out to about $10,250 of context cost on these long sessions. A Viewer model would not remove all of it – SWE-Edit reports its Viewer filters out about 60% of the code it reads. Applying that rate to my own figure gives an estimated saving of about $6,150. The reading cost is measured here; only the 60% filter rate comes from the paper.

5 What to put in the table cell

In long sessions, file-reading (exploration) is 55% of all context tokens – the single biggest cost of the coupling problem. Moving it to a cheaper Viewer model is where the savings are.

Sessions affected: 769 long Claude Code sessions. File-reading accounts for about $10,250 of context cost – 55% of all tool-result cost, the single largest source. Applying SWE-Edit's ~60% filter rate, roughly $6,150 is removable by moving reads to a cheaper Viewer model.

Categories: three needed – Exploration, Execution, Editing (the data's 'action' count merged editing and execution). Editing is only 2% of tokens, so decoupling it improves reliability, not cost. The $10,250 reading cost is measured on this data; only the 60% filter rate is borrowed. Token sizes are approximated and reads are assumed to stay in context, so $6,150 is an upper-ish estimate.